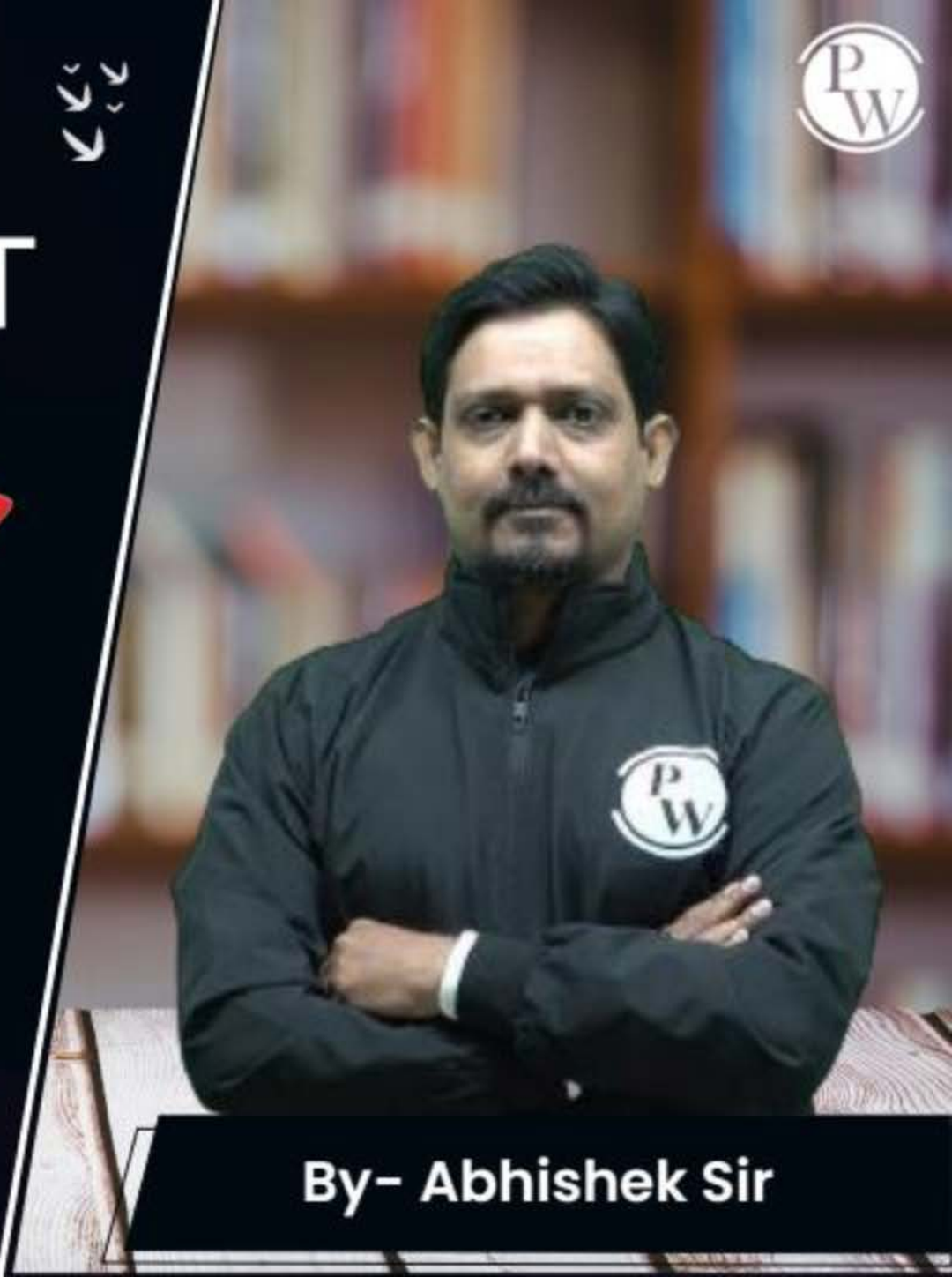
Computer Science & IT



C Programming

Data type & Operator

Lecture No. 06



By- Abhishek Sir

Recap of Previous Lecture



- Topic — Logical operator
- Topic — Bitwise
- Topic — &, |, ~, ^
- Topic
- Topic

Slide

Topics to be Covered



- Topic — << left shift
- Topic — >> right shift
- Topic — ?: conditional
- Topic — printf return value
- Topic

Slide



Associativity Rule

Precedence	Operator	Description	Associativity
1	`++ --`	Suffix/postfix increment and decrement	Left-to-right
	`()`	Function call	
	`[]`	Array subscripting	
	`.`	Structure and union member access	
	`->`	Structure and union member access through pointer	
	`(type){list}`	Compound literal(C99)	
2	`++ --`	Prefix increment and decrement	Right-to-left
	`+ -`	Unary plus and minus	
	`! ~`	Logical NOT and bitwise NOT	
	`(type)`	Type cast	
	`*`	Indirection (dereference)	
	`&`	Address-of	
	`sizeof`	Size-of	
	`_Alignof`	Alignment requirement(C11)	

Slide



Associativity Rule

3	* / %	Multiplication, division, and remainder	Left-to-right
4	+ −	Addition and subtraction	
5	<< >>	Bitwise left shift and right shift	
6	< <= > >=	For relational operators < and ≤ respectively For relational operators > and ≥ respectively	
7	== !=	For relational = and ≠ respectively	
8	&	Bitwise AND	
9	^	Bitwise XOR (exclusive or)	
10	|	Bitwise OR (inclusive or)	
11	&&	Logical AND	
12	||	Logical OR	

Slide



Associativity Rule

13	?:	Ternary conditional	Right-to-Left
14	= += -= *= /= %= <<= >>= &= ^= |=	Simple assignment Assignment by sum and difference Assignment by product, quotient, and remainder Assignment by bitwise left shift and right shift Assignment by bitwise AND, XOR, and OR	
15	,	Comma	Left-to-right

Slide



Bit-wise Operator

Operators	Meaning of operators
&	Bitwise AND
|	Bitwise OR
^	Bitwise exclusive OR
~	Bitwise complement
<<	Shift left
>>	Shift right



Bit-wise Operator

What is the output the program

```
#include <stdio.h>
    int main ()  {
      int x = 5, y=17, z
      z = x&y;
      printf("%d", z);
 }
```

0 0 0 1 0 1

0 1 0 0 0 1

(A) 1 ✓

(B) 21

(C) 2

(D) -6

Slide



Bit-wise Operator

What is the output the program

```
#include <stdio.h>
    int main ()  {
      int x = 5, y=17, z
      z = x|y;
      printf("%d", z);
 }
```

(A) 1
(B) 21
(C) 2
(D) -6

```
  000101
  100001
--------
  100101
```

Slide



Bit-wise Operator

What is the output the program

```
#include <stdio.h>
    int main ()  {
      int x = 5, y=17, z
      z = x|y;
      printf("%d", z);
 }
```

(A) 1 **(B) 21**

(C) 2 (D) -6



Bit-wise Operator

What is the output the program

```
#include <stdio.h>
    int main ()  {
      int x = 5, y=17, z
      z = x^y;
      printf("%d", z);
 }
```

(A) 1 (B) 21

(C) 20 (D) -6

000101
100001
100100

Slide



Bit-wise Operator

What is the output the program

```
#include <stdio.h>
    int main ()  {
      int x = 5, y=17, z
      z = x^y;
      printf("%d", z);
 }
```

(A) 1

(B) 21

(C) 20

(D) -6

Slide



Bit-wise Operator

What is the output the program

```
#include <stdio.h>
    int main ()  {
        int x = 5, z
        z = ~x;
        printf("%d", z);
    }
```

(A) 1
(B) 21
(C) 20
(D) -6

$$\sim X = -(X+1)$$

$X = 5 : -(5+1) = -6$

$X = -5 : -(-5+1)$

$: -(-4) = 4$

Slide



Bit-wise Operator

What is the output the program

```
#include <stdio.h>
    int main ()  {
        int x = 5, z
        z = ~x;
        printf("%d", z);
    }
```

(A) 1

(B) 21

(C) 20

(D) -6

Slide



Bit-wise Operator

```
#include<stdio.h>
   int main(){
            char a = 8;
            int k;
            k =a<<3;
            printf("%d", k);
            return 0;
   }
```

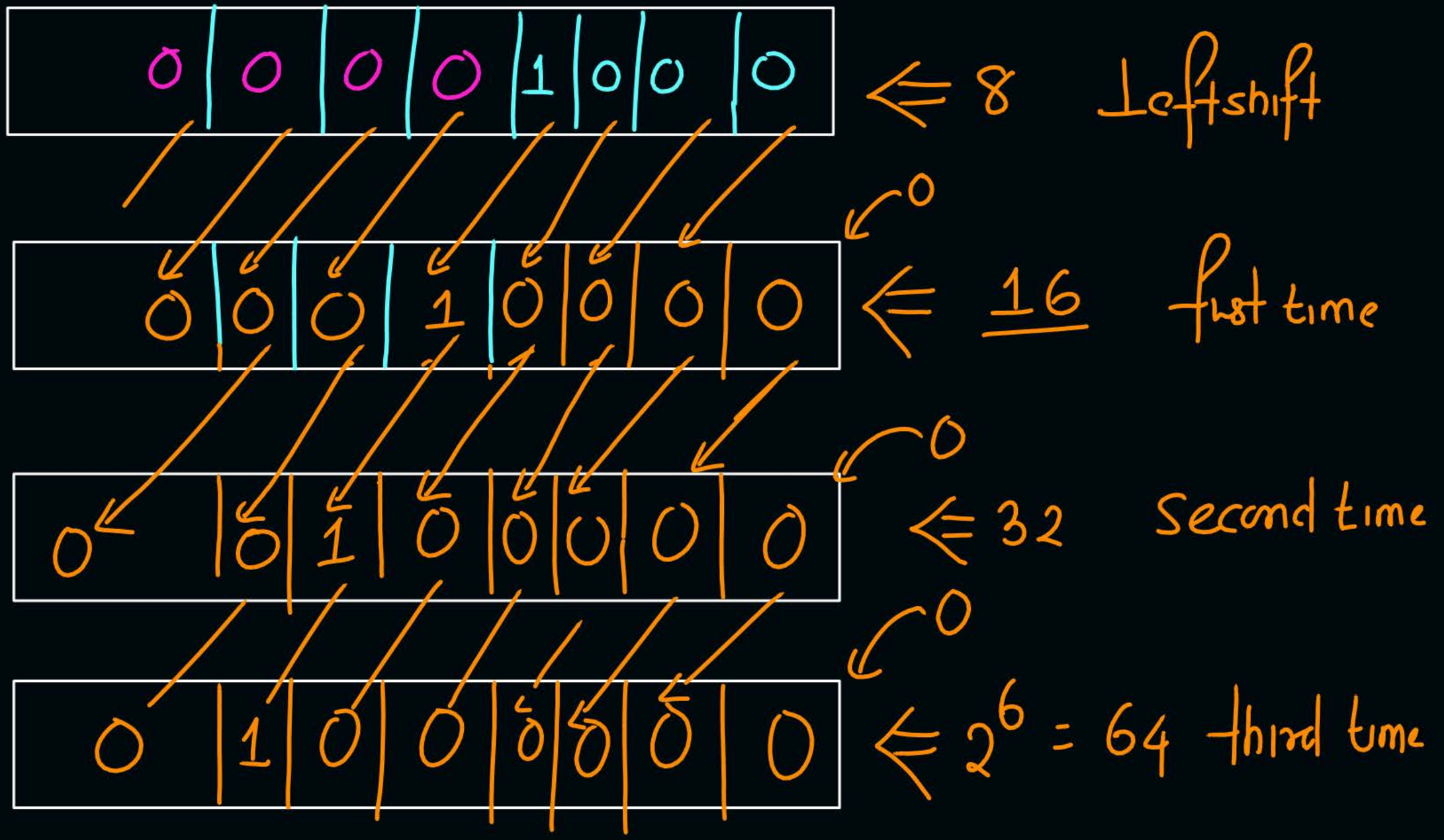
(A) 1
(B) 64
(C) 20
(D) -6

<< Shift left

$a << k = a * 2^k$

Each shift left operation is Equivalent to multiply by 2

Slide


0 0 0 0 1 0 0 0
<< 8
leftshift
0 0 0 1 0 0 0 0
0
<< 16
fust time
0 0 1 0 0 0 0 0
0
<< 32
second time
0 1 0 0 0 0 0 0
0
<< 2^6 = 64
third time



Bit-wise Operator

```
#include<stdio.h>
   int main(){
            char a = 8;
            int k;
            k =a<<3;
            printf("%d", k);
            return 0;
   }
```

(A) 1 **(B) 64**

(C) 20 (D) -6

Slide



Bit-wise Operator

```
#include<stdio.h>
   int main(){
            char a = 64;
            int k;
            k =a>>3;
            printf("%d", k);
            return 0;
   }
```

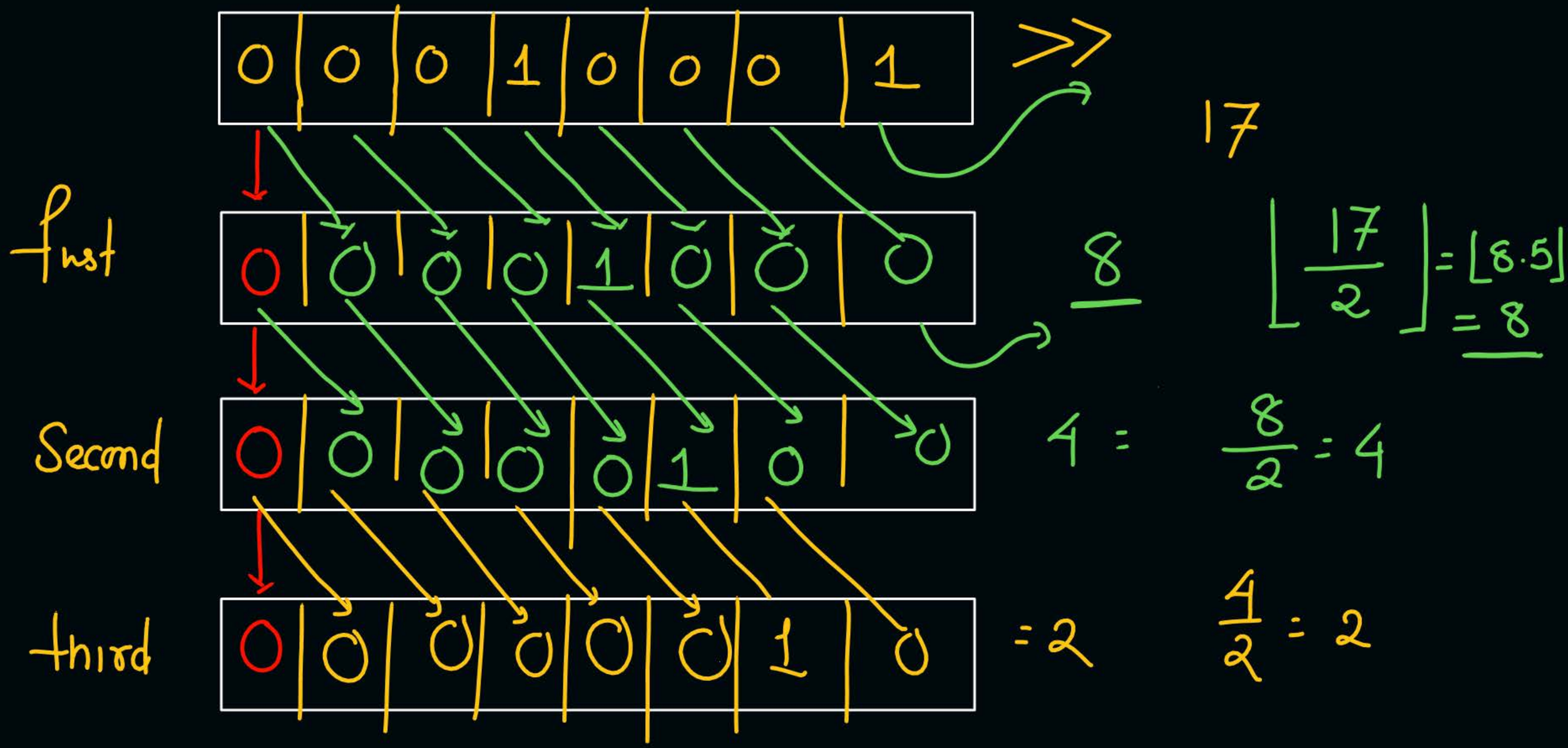
8 bit

$$\frac{64}{2^3} = \frac{64}{8} = 8$$

(A) 1 (B) 21

(C) 8 (D) -6

Slide


0 0 0 1 0 0 0 1
>>
17
first
0 0 0 0 1 0 0 0
8
⌊17/2⌋ = ⌊8.5⌋ = 8
Second
0 0 0 0 0 1 0 0
4 = 8/2 = 4
third
0 0 0 0 0 0 1 0
= 2
4/2 = 2

Each right shift is equivelent to divide by = 2

$$\frac{17}{2^3} = \left\lfloor \frac{17}{8} \right\rfloor = 2$$

$$a >> k = \left\lfloor \frac{a}{2^k} \right\rfloor$$



Bit-wise Operator

```
#include <stdio.h>
    int main ()  {
      int x = 5,y=24, z;
          z = ~x + y >> 2;
       printf("%d", z);
    }
```

(A) 0
(B) 4
(C) 12
(D) -6

first Arithmetic then shift

$\sim x + y >> 2$

$-6 + 24 >> 2$

$18 >> 2 = \left\lfloor \frac{18}{2^2} \right\rfloor = 4$

Slide



Toipc:Question

```
#include <stdio.h>
   int main () {
   int a =(( 5+6!=10 )==8)&&((6*2/4>10)==(10>6));
   printf("%d", a);
   return 0;
   }
```

0

short

11 != 10

The output of the program is ____

= (1 == 8)

= 0

Slide



Toipc:Question

```
#include <stdio.h>
     int main ()  {
        int a =5+5!=10|| 6+4;
     printf("%d", a);
    }
```

The output of the program ___1___

5+5!=10

10!=10 ||

0 || 6+4

0||10 ← Non zero

Slide



Question

```
#include <stdio.h>
int main() {
        char ch = 65+13;
        printf("%c", ch);
        return 0;
}
```

'a'+13
= 97+13
= 110

65 - A
66
67
68
69
70
71
72
73

74
75
76
77
78 — N

a - 97
A - 65
'0' - 48

(a) A
(B) N
(c) n
(d) z

Slide



Bit-wise Operator

```
#include <stdio.h>
    int main ()  {
      int x = 5,y=24, z;
          z = ~ x + ~y ;
       printf("%d", z);
    }
```

(A) -6

(B) -24

(C) 31

(D) -31

$\sim x = -(x+1) = -(24+1)$

$= -25$

$\sim 5 = -6$

$-6 - 25 = -31$

Slide



Bit-wise Operator

```
#include <stdio.h>
    int main ()  {
      int x = 5,y=24, z;
          z = ~ x + ~y ;
       printf("%d", z);
    }
```

(A) -6

(B) -24

(C) 31

(D) -31



Bit-wise Operator

```
#include <stdio.h>
    int main ()  {
      int x = 5,y=24, z=10;
          z = x & y | z ;
       printf("%d", z);
    }
```

(A) 10
(B) -5
(C) 24
(D) 11

x 0 0 0 1 0 1 5

y 0 1 1 0 0 0 24

0 0 0 0 0 0

0 0 0 0 0 0 0

0 0 1 0 1 0 z

0 0 1 0 1 0

Slide



Bit-wise Operator

```
#include <stdio.h>
    int main ()  {
      int x = 5,y=24, z=10;
          z = x & y | z ;
       printf("%d", z);
    }
```

(A) **10**

(B) -5

(C) 24

(D) 11



Bit-wise Operator

```
#include <stdio.h>
    int main ()  {
      int x = 5,y=24, z=10;
          x = x & 10;
          y = y | 10;
          z = z >> 2;

       printf("%d", x+y+z);
    }
```

Output of the following program is ____

000101
001010
000000

x = 0

011000
001010
011010
(26)

y = 26

10 >> 2 : $\lfloor \frac{10}{4} \rfloor$: 2

0+26+2 : 28

Slide



Bit-wise Operator

```
#include <stdio.h>
    int main ()  {
      int x = 5,y=24, z=10;
          x = x & 10;
          y = y | 10;
          z = z >> 2;

       printf("%d", x+y+z);
    }
```

Output of the following program is **28**____



Question

```
#include <stdio.h>
int main()     {
        int num1 = 5;
        int num2 = 3;
        int num3 = 2;
        num1 = num2++ * --num3 / ++num1;
        printf( "%d %d %d", num1, num2, num3);
        return 0;
    }
```

A. 0 4 1
B. 6 4 1
C. 6 3 2
D. 0 3 2



++ & -- Operator

Consider the following C program.

```
##include<stdio.h>
int main () {
    int m=20, k=50;
    int n, n1;
    n=++m + ++k;
    n1=m-- + --k ;
    n--;
    --n1;
    n-=n1;
    printf("%d", n+k);
    return 0;
}
```

The output of the program is _____

k: 50 51 50

m: 20 21 20

n: 21 + 51 = 72, 71

n1: 21 + 50 = 71 70

n -= n1 : n = n - n1

n = 71 - 70 = 1

1 + 50 = 51

(51)

Slide



Question

```
#include <stdio.h>
int main()     {
        int num1 = 5;
        int num2 = 3;
        int num3 = 2;
        num1 = num2++;
        num2 = --num3;
        return 0;
}
```

(A) 6

(B) 5

(c) 4

(D) 10

num2 = 4

num3 = 1

num1 = 3 ← post

num2 = 1

3 + 1 + 1

The valur of num1+num2+num3 ____(5)____

Slide



Question

```
#include <stdio.h>
int main()     {
        int num1 = 5;
        int num2 = 3;
        int num3 = 2;
        num1 = num2++;
        num2 = --num3;
        return 0;
    }
```

printf("%d", num1+num2+num3)

5

The valur of num1+num2+num3_____. Ans 5

Slide



Question

HW

Consider the following C program.

```
#include<stdio.h>
int main () {
      int a=2,b=5,c;
      c=a+++b;
      printf("%d",a+b+c);
      return 0;
}
```

The output of the program is _____



Toipc: Scope of a variable

The scope of a variable in C is the block or the region in the program where a variable is declared, defined, and used. Outside this region, we cannot access the variable and it is treated as an undeclared identifier.



Toipc:Scope of variable

```
#include<stdio.h>
int   main {
      {
            int a =10 ;
            printf(“%d”; a); // will print the 10
      }


}
```



Toipc:Scope of variable

```
#include<stdio.h>
int  main() {
   {
    int a =10 ;
   }
   printf("%d", a); // will not print 10
   return 0 ;
}
```

Error a is Not defind



Toipc:Scope of variable

```
#include<stdio.h>
int  main() {
   {
          int a =10 ;
          printf("%d", a); // will print 10
   }

   return 0 ;
}
```

```c
#include<stdio.h>
int x = 40;
int  main() {
       int x = 30;
      {
          int x = 20;
          {
                int x = 10;
               printf("%d", x);
          }
        }
       return 0 ;
}
```

→ 10

Slide

```c
#include<stdio.h>
int x = 40;
int  main() {
       int x = 30;
      {
          int x = 20;
          {
              //int x = 10;
              printf("%d", x);
          }
       }
      return 0 ;
}
```

20

Slide

```c
#include<stdio.h>
int x = 40;
int  main() {
       int x = 30;
      {
        //int x = 20;
          {
             //int x = 10;
               printf("%d", x);
          }
       }
      return 0 ;
}
```

30

Slide

Static scoping - (C)

```c
#include<stdio.h>
int x = 40;
int  main() {
    // int x = 30;
      {
        //int x = 20;
          {
            //int x = 10;
              printf("%d", x);
          }
       }
      return 0 ;
}
```

← global variable

// 40

if a variable definition is Not found then definition of variable searched in next upper block this process repeated until definition of variable is found

Slide



Print() Return value

printf("ABCD");

work = ABCD print

return value = 4

return. integer

No. of character printed

Slide



Print() Return value

```
#include<stdio.h>
int main(){
   printf("%d",printf("ABCD"));
   return 0;}
```

Inner will execute

ABCD4

(A) ABCD4
(B) 4
(C) ABCD 4
(D) ABCD

outer print 4

Slide



Print() Return value

```
#include <stdio.h>
  int main(){
     printf("\n%d", printf("ABCD"));
     return 0;
   }
```

ABCD

4 → 4 ← next line

New Line \n New line character

Slide



Print() Return value

```
#include <stdio.h>
  int main(){
     printf("\n%d", printf("ABCD"));
     return 0;
   }
```

```
ABCD
4
```



Print() Return value

```
#include <stdio.h>
  int main(){
     int a=1000;
     printf("%d",printf("\n%d",a));
     return 0;
}
Output of the program is _____
(A) 1005
(B) 10005
(C) 1000
(D) 1000
    5
```

Control

Next line 1000

4 character

+1

New Line character

Slide



2 mins Summary

Topic — Shift left & shift Right

Topic — practice

Topic — Scope of variable

Topic — printf return value.

Topic

Slide

THANK - YOU

